# Lecture 5: Procedures

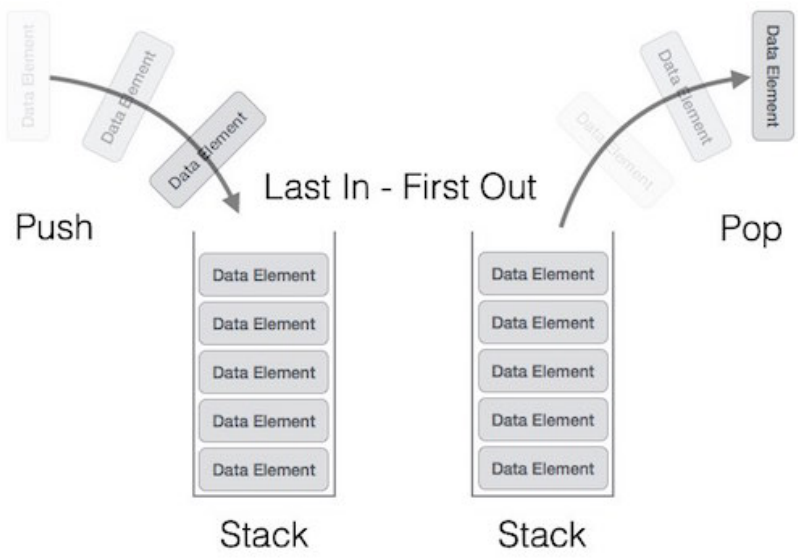
## Table of contents

# 1 Objectives:

- Understand the concept and operation of the run-time stack.
- Learn how to define a procedure and how to call it.
- Explore how the CALL and RET instructions manipulate the stack and program flow.

# 2 Stack Operations

- A stack is a data structure in computing, often called a **LIFO** (Last-In, First-Out) structure because the last value put into the stack is always the first value taken out.
- A stack is associated with two operations: **Push** and **Pop**.
  - Push is an operation to add an element into the stack (at the top)
  - Pop is an operation to remove one element from the stack (from the top)




## 2.1 The Run-time Stack

- Every process (a running program) has a special region of memory called the ***run-time stack***. This stack is managed directly by the CPU and is crucial for calling procedures, passing arguments, and storing local variables.
- In 32-bit mode, the ESP register (known as the stack pointer) always holds the memory address of the item currently at the top of the stack.
- The stack grows **downwards** in memory, from higher addresses to lower addresses. This means that when you `PUSH` data. `ESP` decrements.
- ESP can be manipulated by several CPU instructions, such as `PUSH`, `POP`, `CALL` and `RET`.

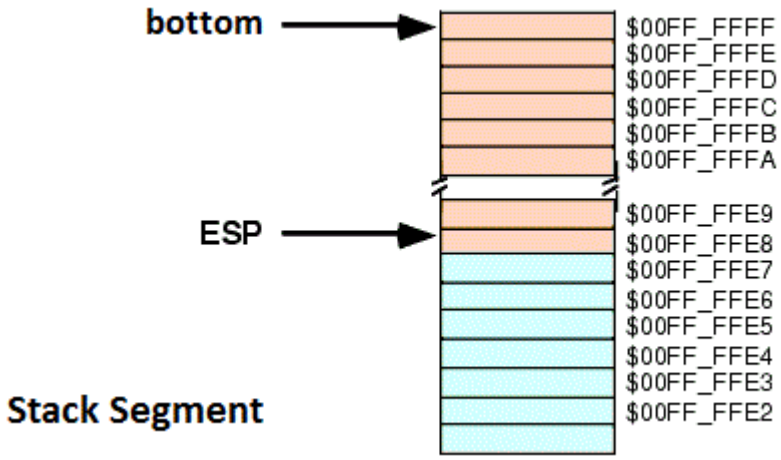



Figure 1: ESP points to the top of the stack. The area above ESP (in light red) is inside the stack. The area below the ESP (light blue) is empty.

## 2.2 PUSH Instruction

**Syntax**

```
PUSH    <source>
```

The PUSH instruction adds data to the top of the stack. The source operand can be either 16-bit or 32-bit value.

### 2.2.1 How It works:

1. First, it decrements the stack pointer (ESP) to make room for the new item.
    - if the source operand is 32-bit ('DWORD'), the stack pointer is decremented by a value of 4
    - if the source operand is 16-bit (WORD), the stack pointer is decremented by a value of 2.

2. It copies the value from the source operand to the memory location now pointed to by ESP.

### 2.2.2 The valid formats are:

```
push    reg/mem32
push    reg/mem16
push    imm32
```

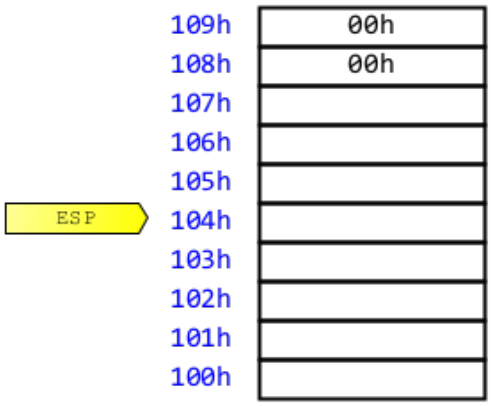
### 2.2.3 Example:

Let's visualize the following instruction.

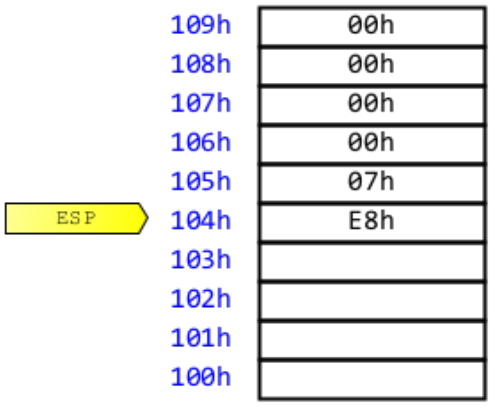
```
push    2024    ; 2024 => 000007E8h
```

1. Let us consider the following run-time stack as shown below; before executing push instruction:


2. When push 2024 is executed, the first step is to decrement ESP by 4.




3. Then, the value `2024` , which will be treated as `imm32`, will be copied onto the stack at the location referenced by `ESP`:




4. The above diagram shows the stack after pushing a total of four bytes.

> **Note**
>
> The area of the stack below ESP (at lower addresses) is logically empty, and will be overwritten the next time the current program executes any instruction that pushes a value on the stack.

## 2.3 POP Instruction

**Syntax**

```
POP    <destination>
```

The POP instruction removes data from the top of the stack. The destination operand can be of size 16-bit or 32-bit

### 2.3.1 How it works:

1. First, it **copies** the value from the memory location at the top of the stack (pointed by ESP) into the destination operand. The number of bytes to be copied is determined by the size of the destination operand.
    - If the size of destination operand is 16-bit, then two bytes are transferred into the destination operand.
    - If the size of destination operand is 32-bit, then four bytes are transferred into the destination operand
2. Second, it **increments** ESP by the size of the operand (2 for 16-bit, 4 for 32-bit), effectively "removing" the data by moving the stack pointer past it.

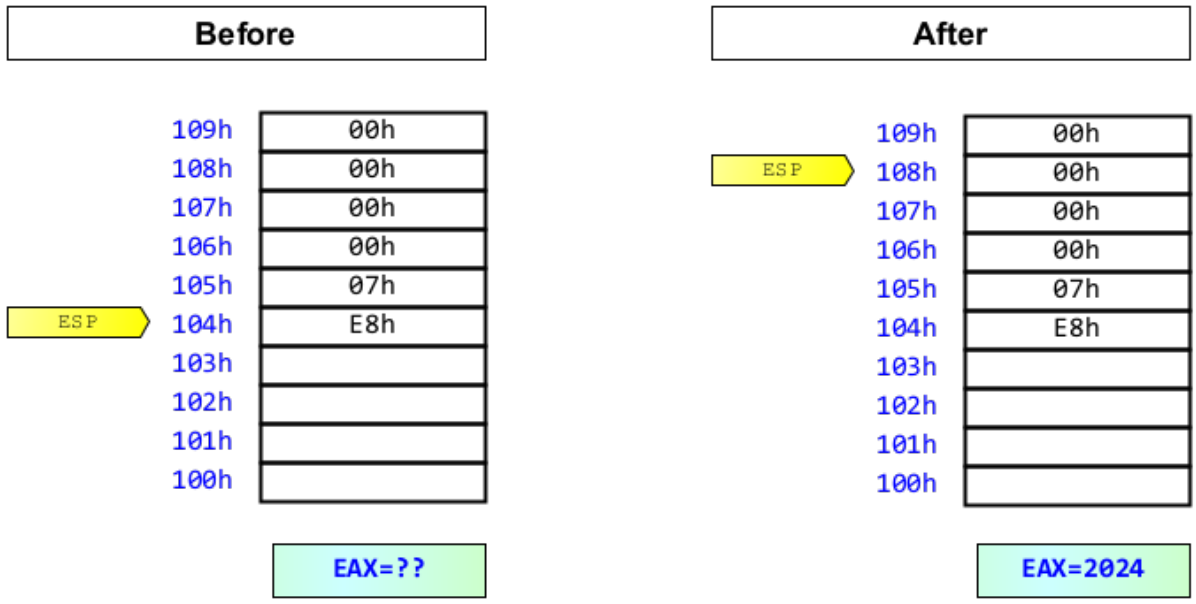
### 2.3.2 Valid Operands:

```
pop    reg/mem16
pop    reg/mem32
```

### 2.3.3 Example:

Let us consider the previous state of the run-time stack. The following instruction pop the top value (which is 2024), and put it into EAX register:

```
pop   eax
```




### 2.3.4 Is Popped Data Erased?

No! When you POP a value, the data remains in memory. However, it's now in the "empty" part of the stack (at an address higher than ESP). It will be overwritten by the next PUSH operation. The stack is "blind" and only considers data at ESP and above to be valid.

### 2.3.5 Flags Affected

Both push and pop instructions have no effect on EFLAGS register.

## 2.4 Saving the Machine State

Sometimes, you need to save the entire state of the general-purpose registers, especially before calling a procedure that might modify them.

### 2.4.1 `PUSHA` and `POPA` Instructions

**Syntax**

```
PUSHA        ; no operand

POPA         ; no operand
```

- `PUSHA` (push all) pushes the contents of the 16-bit general-purpose registers onto the stack. The registers are stored on the stack in the following order: AX, CX, DX, BX, SP, BP, SI, and DI.
- `POPA` (pop all) pops words from the stack into the 16-bit general purpose registers in the following order: DI, SI, BP, BX, DX, CX, and AX. The value on the stack for SP register is ignored.

### 2.4.2 `PUSHAD` and `POPAD` Instructions

**Syntax**

```
PUSHAD       ; no operand

POPAD        ; no operand
```

- `PUSHAD` (push all doublewords) pushes the contents of the 32-bit general-purpose registers onto the stack. The registers are stored on the stack in the following order: EAX, ECX, EDX, EBX, ESP (original value), EBP, ESI, and EDI.

- POPAD (pop all doublewords) pops doublewords from the stack into the 32-bit general purpose registers in the following order: EDI, ESI, EBP, EBX, EDX, ECX, and EAX. The value on the stack for ESP register is ignored.

### 2.4.3 PUSHFD and POPFD Instructions

**Syntax**

```
PUSHAD          ; no operand

POPAD           ; no operand
```

- PUSHFD (push flag doubleword) pushes the contents of the 32-bit EFLAGS register onto the stack.
- POPFD (pop flag doubleword) pops a doubleword from the stack into the 32-bit EFLAGS register.

## 2.5 Practical Stack Applications

The stack isn't just a theoretical concept; it's used constantly for practical programming tasks.

### 2.5.1 Reversing a String

Because the stack is a LIFO structure, it's perfect for reversing sequences. The code below first traverses through a string (msg) and pushes each character onto the stack. It then pops the letters from the stack and stores them back into the same string variable, resulting in a perfect reversal.

```
        section .data
msg     db      "Hello World!", 0

        section .text
_main:
        ; --- Push characters onto the stack and count them ---
        mov     esi, 0    ; set index to 0
L1:
        movzx   ax, BYTE [esi + msg]
        cmp     al, 0
        je      ENDL1
        push    ax
        inc     esi
        jmp     L1
ENDL1:
        ; At this point, ESI holds the length of the string

        ; --- Pop characters back into the string in reverse order ---
        mov     ecx, esi        ; Use the length as the loop counter
        mov     esi, 0          ; Reset index to start of string
L2:
        pop     ax              ; Pop the word
        mov     [msg + esi], al ; Store the character back
        inc     esi
        loop    L2
```

Let us examine the following instruction:

```
        movzx     ax, BYTE [esi + msg]
```

We can't directly `PUSH` a single 8-bit character onto the stack. We need a way to "package" our 8-bit character into a 16-bit container before pushing. That's what `movzx` does. It takes the 8-bit character and puts it into the lower half of `ax` (the `al` register). Then, it fills the upper half (`ah`) with zeros.

### 2.5.2 Memory to Memory Operations

The stack provides an easy way to move data between two memory locations or swap them without needing an extra register.

- Let translate the following statement into assembly language using run-time stack:

```
y = x;      /* assign x to y */
```

```
      push  DWORD[x]
      pop   DWORD[y]
```

- Let us also swap two values `x` and `y` using run-time stack:

```
      push  DWORD[x]
      push  DWORD[y]
      pop   DWORD[x]
      pop   DWORD[y]
```

### 2.5.3 Preserving Loop Counters in Nested Loop

When you have nested loops, the outer loop's counter (`ECX`) will be overwritten by the inner loop. The stack is the perfect place to save it.

```
      mov   ecx, 100    ; outer loop index
OUTER:
      push  ecx         ; store outer loop index
      mov   ecx, 20
INNER:
      .
      .
      loop  INNER

      pop   ecx         ; restore outer loop index
      loop  OUTER
```

# 3 Procedures

- Complex problems are best solved by breaking them into smaller, manageable tasks. In programming, this is achieved with **procedures** (also known as **functions** or **subroutines**)
- In NASM, a procedure is simply a block of code ending with `RET` instruction., identified by a label.
- NASM has no built-in directive to support procedures. However, MASM, the other assembler, has built-in directive: `PROC` and `ENDP`.

> **Recall: The Instruction Pointer**
>
> Remember that the CPU fetches instructions from the memory location pointed to by the `EIP` register, also known as the **Program Counter (PC)**. After fetching an instruction, the PC is automatically incremented. This means that by the time an instruction is executed, the PC is already pointing to the *next* instruction in memory.

## 3.1 `CALL` and `RET` Instructions

`CALL` and `RET` are the two instructions that make procedure possible. They work together using the stack to manage the flow of your program.

**Syntax**

```
        CALL    <procedure_label>
        ;......

<procedure_label>:                      ; beginning of procedure
        ; code for the procedure
        ; ...
        RET                             ; end of procedure
```

Here is the magic behind them:

- `CALL` **Instruction**: When the CPU executes `CALL <label>` , it performs two crucial steps:
  - it pushes the current value of PC (i.e., `EIP`) onto the run-time stack. Then,
  - It jumps to the target address defined by `<label>`.
- The `RET` instruction simply pops from stack into the PC. Therefore, the EIP register will execute the instruction that follows the `CALL` instruction.

**Example 3.1.** In the following example, the procedure `foo` sets EAX, EBX, ECX and EDX to zero. In the main program, we call this procedure to initialize data registers to zero.

```
foo:
      xor   eax, eax
      xor   ebx, ebx
      xor   ecx, ecx
      xor   edx, edx
      ret                ; end of initialize procedure

_main:
      call  foo
      .
      .
      .
      ret                ; end of main program
```

### 3.1.1 Visualizing the `CALL` and `RET` Flow

The following diagrams show how the stack and the `EIP` register work together during a `CALL` and `RET`.

- In Figure 2, the program is fetching $instruction_1$ , which is located at address $4000$.

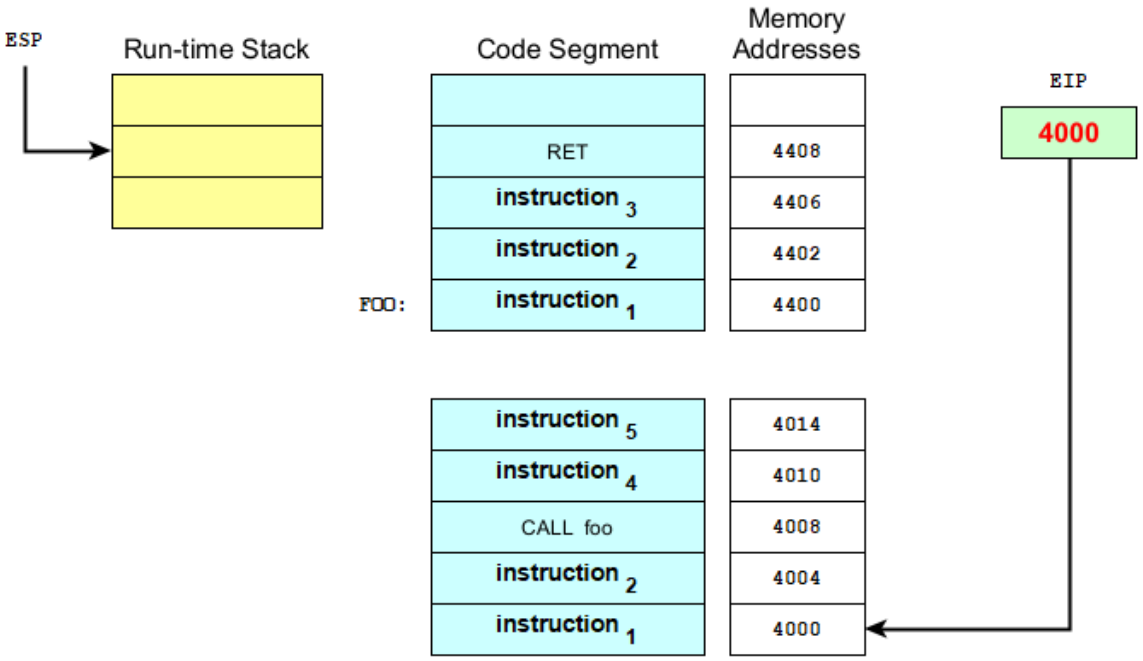



Figure 2

- When the CPU starts to execute $instruction_1$ , the EIP is pointing to $instruction_2$, as shown in Figure 3. After instruction execution, the CPU will fetch $instruction_2$.
- When the CPU starts to execute $instruction_2$, the EIP is incremented and pointing to the instruction at location $4008$, as shown in Figure 4.
- After executing $instruction_2$, the CPU will fetch `CALL foo` instruction. Before executing this instruction, the EIP is incremented to $4010$, as shown in Figure 5.
- The `CALL` instruction, when executed, will perform the following two operations:
  - first, It pushes the current value of EIP onto the stack.
  - second, It replaces the value of EIP with the offset address `FOO` (which has the value $4400$). So, the CPU will branch to that address, as illustrated in Figure 6.

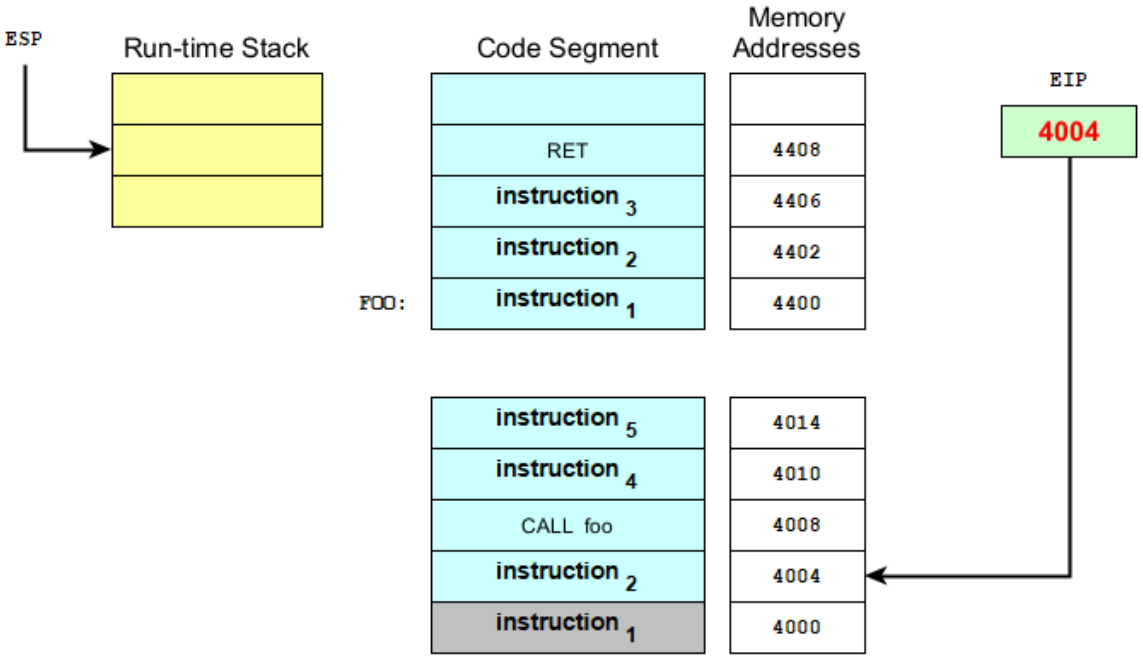



Figure 3

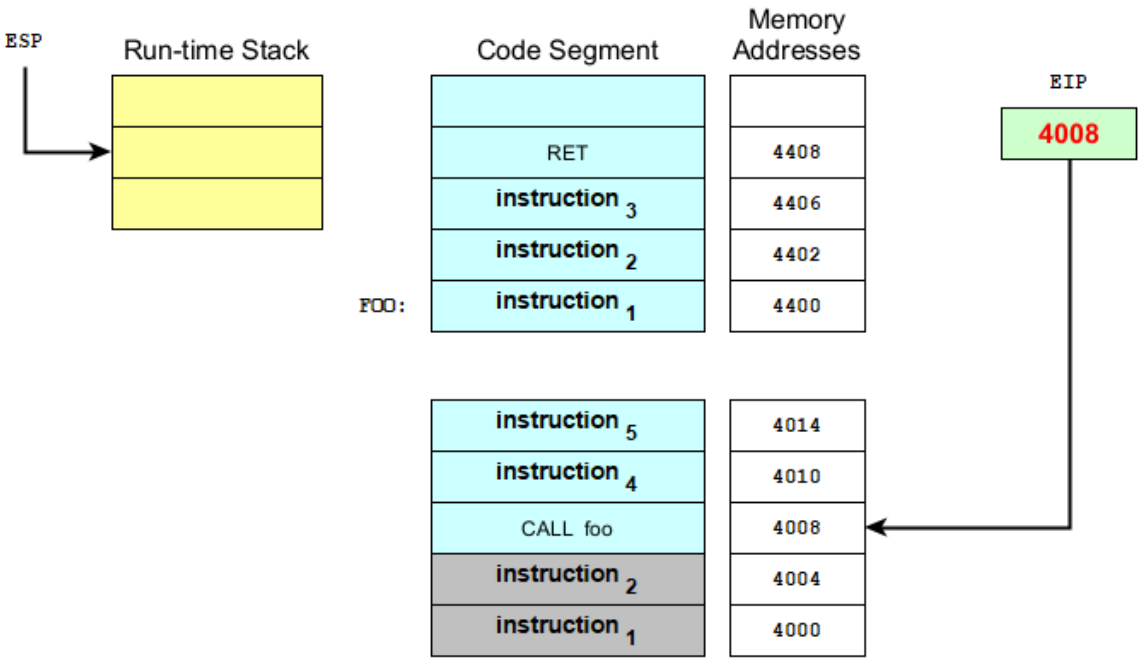



Figure 4

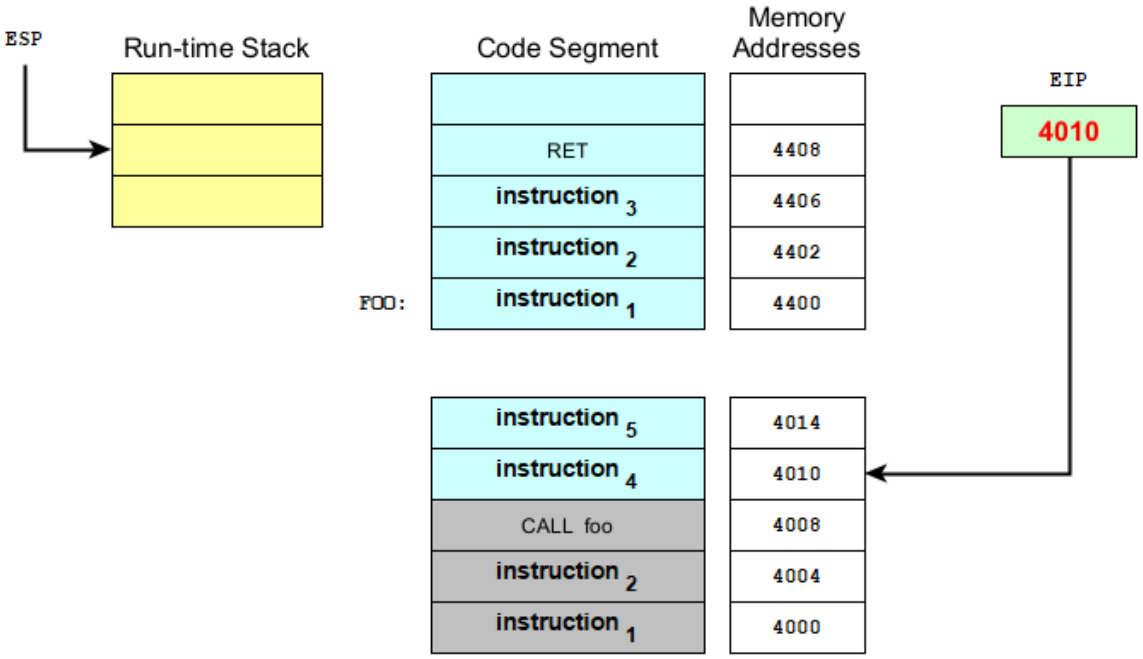



Figure 5

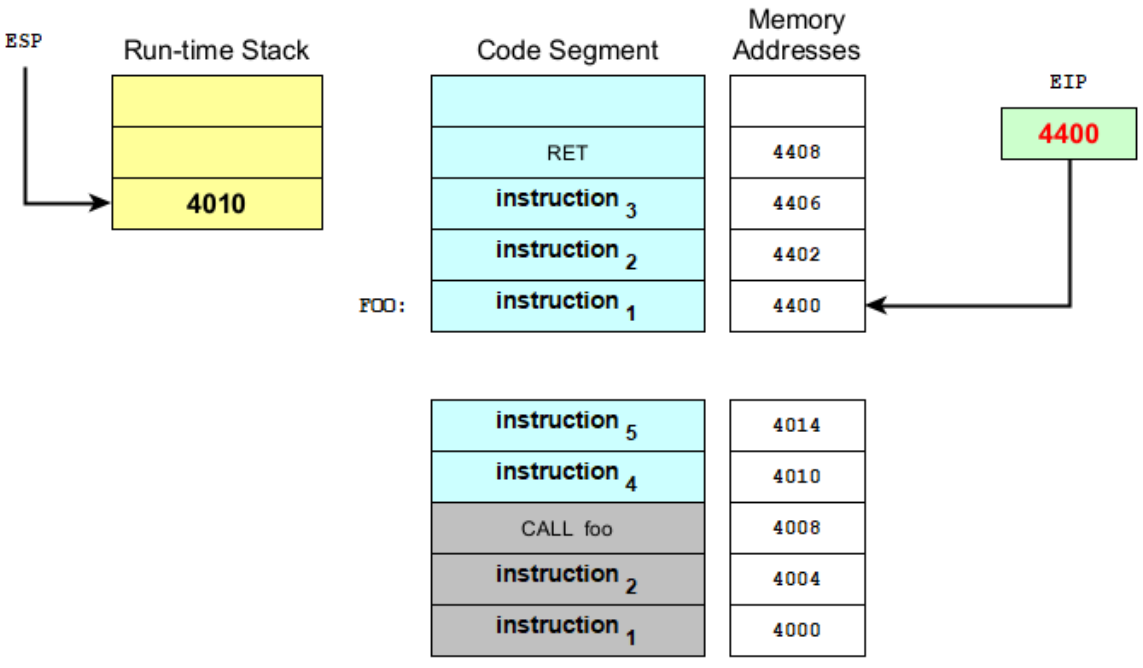



Figure 6

- Logically, the CPU will start to execute the procedure `FOO`.
- The CPU will execute $instruction_1$, $instruction_2$, and $instruction_3$ of procedure `FOO`, which are located at $4400$, $4402$, and $4406$, respectively.
- After executing $instruction_3$ in procedure `FOO`, the EIP is pointing to the next instruction, which is `RET`. See Figure 7.

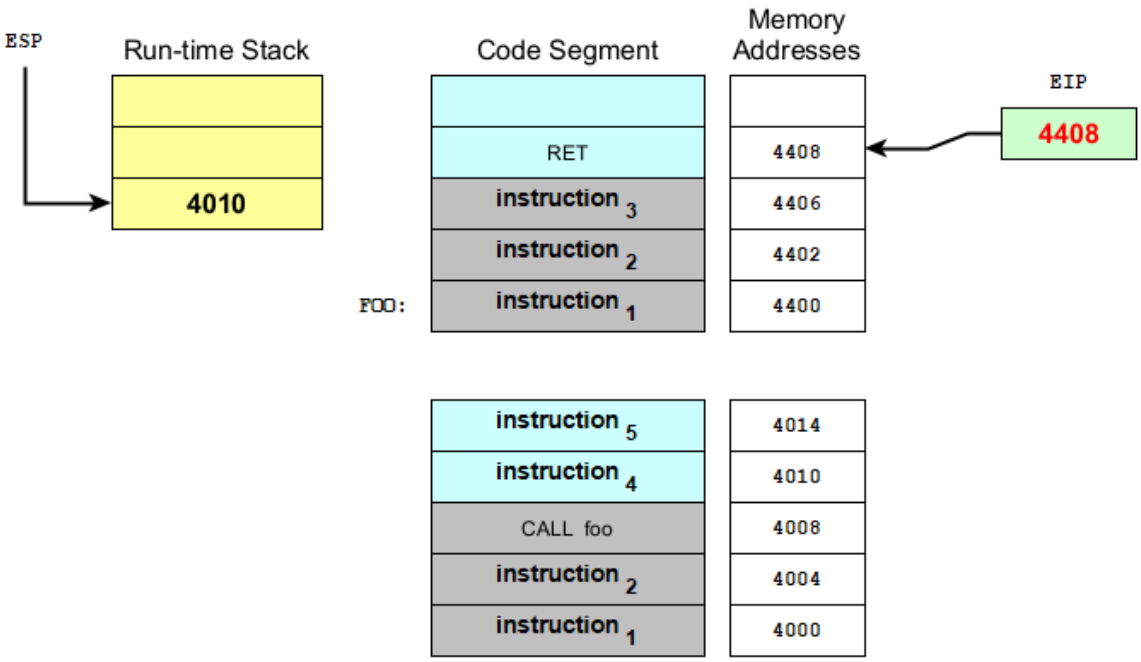



Figure 7

- When the CPU executes `RET` instruction, it will implicitly perform the following instruction: `pop EIP`. This implies that the value in EIP will be replaced with the value pointed by `ESP`. Then, the `ESP` will be incremented by 4. as shown in Figure 8. Consequently, the CPU will branch back to the instruction that follows the `CALL` instruction.

## 3.2 Security Warning

The `CALL`/`RET` mechanism is powerful, but it relies on the return address on the stack remaining intact. What if we could maliciously overwrite it? This is the

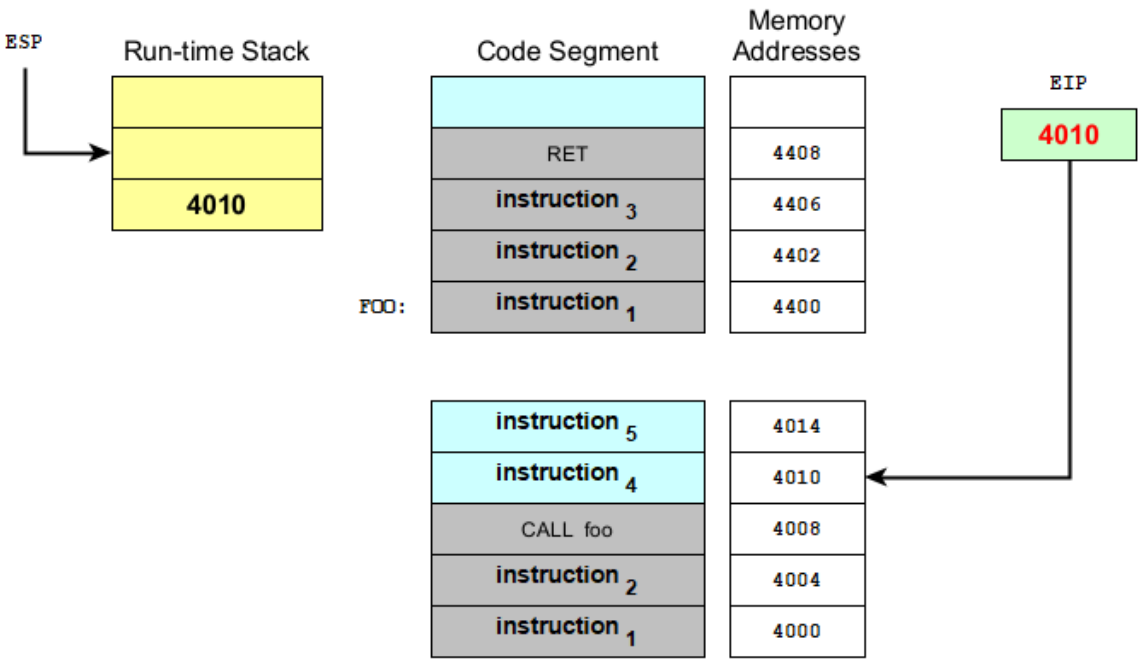



Figure 8

basis of a famous attack called a **stack buffer overflow** or **stack smashing**.

### Stack Buffer Overflow

The EIP register cannot be accessed directly by software; it is controlled implicitly by control-transfer instructions (such as JMP, Jcc, CALL, and RET), interrupts, and exceptions. The only way to read the EIP register is to execute a CALL instruction and then read the value of the return instruction pointer from the procedure stack. The EIP register can be loaded indirectly by modifying the value of a return instruction pointer on the procedure stack and executing a return instruction (RET or IRET).
A stack buffer overflow can be caused deliberately as part of **stack smashing**. If the run-time stack is filled with data supplied from an *untrusted user* then that user can corrupt the stack in such a way as to **inject executable code** into the running program and take control of the process. This is one of the oldest and more reliable methods for attackers to gain unauthorized access to a computer.

> Here is a link of a recent discovered attack that exploits stack-based buffer overflow. Click here

## 3.3 Nested Procedure Call

- A called procedure can call another procedure before the first procedure returns. This is known as *nested procedure call*.
- Suppose the `main` procedure calls a procedure named `Sub1`. While `Sub1` is executing, it calls `Sub2` procedure. While `Sub2` is executing, it calls the `Sub3` procedure. The process is shown in Figure 9.

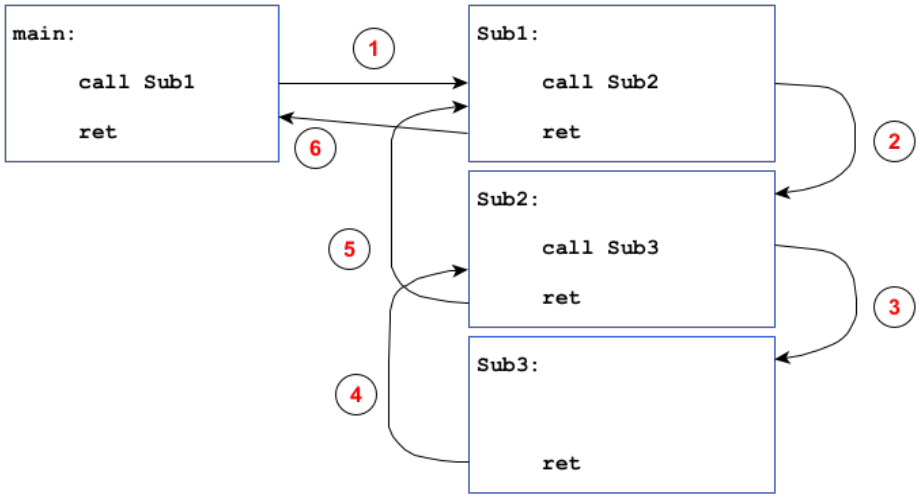



Figure 9

## 3.4 Passing Register Arguments to Procedures

Hard-coding variable names inside a procedure makes it non-reusable. A much better design is to pass data to the procedure as **arguments**. In assembly, the fastest way to do this is through registers. Let's consider the following scenario:

- Suppose we have two variables (as vectors): `vec1` and `vec2`. The size of the two vectors are defined in a constant literal named `VECSIZE`.

- The procedure `addVectors` adds two vectors (i.e., `vec1` + `vec2`) and stores the resulted vector in `vec1` (i.e., `vec1 = vec1 + vec2`) Here is the code:

```
addVectors:
      mov   esi, 0          ; index counter
      mov   ecx, VECSIZE
top:
      mov   eax, [esi*4 + vec1]
      add   eax, [esi*4 + vec2]
      mov   [esi*4 + vec1], eax
      inc   esi
      loop  top
```

- If you call the procedure, it could only add two specific vectors (`vec1` and `vec2` ONLY). If we have another two vectors, say `vec3` and `vec4`, then you need to write another procedure.
- It's not a good practice to include references to specific variable names inside the procedure .
- A better approach is to pass the offset address of an array to the procedure and pass an integer specifying the number of array elements . Here is a better solution:

```
addVectors:
      push    edi           ; EDI is pointing to the destination vecto
      push    esi           ; EDI is pointing to the source vector
      push    ecx           ; ECX has the number of elements in the tu
top:
      mov     eax, [edi]
      add     eax, [esi]
      mov     [edi], eax
      add     esi, 4        ; goto next element
      add     edi, 4        ; goto next element
      loop    top

      pop     ecx           ; restore ECX
      pop     esi           ; restore ESI
```

```
    pop     edi         ; restore EDI
    ret
```

- The registers ESI, EDI and ECX are acting as ***arguments***.
- In the main program, you should call the procedure as follows:

```
    mov   edi, vec1     ; destination vector (first argument)
    mov   esi, vec2     ; source vector (second argument)
    mov   ecx, VECSIZE  ; Array size (third argument)
    call  addVector
```

- In assembly language, it is common to pass arguments inside general–purpose registers.

### 3.4.1 Saving and Restoring Registers

- In the addVector example, ECX, EDI and ESI were pushed on the stack at the beginning of the procedure and popped at the end.
- A well-behaved procedure should **save** any registers it plans to modify (using PUSH or PUSHAD) at the beginning and **restore** them (using POP or POPAD) right before it returns.
- The exception to this rule pertains to registers used as return values, usually EAX. Do not push and pop them.

## 3.5 Documenting Procedures

- It is a good practice to document each procedure you have created. The document should contain at least:
  1. A description of all tasks accomplished by the procedure.
  2. **Receives**: A list of parameters; state their usage and requirements.
  3. **Returns**: A description of values returned by the procedure.

4. **Requires**: Optional list of requirements called preconditions that must be satisfied before the procedure is called.

- Here is an example:

```
;-----------------------------------------------------
; Calculates and returns the sum of three 32-bit integers.
;
; Receives : EAX, EBX, ECX, the three integers. May be signed
;            or unsigned
; Returns  : EAX = sum, and the status flags (Cary, Overflow,
;            etc) are changed.
; Requires : nothing
;-----------------------------------------------------

SumOf:                  ; beginning of procedure
        add eax, ebx
        add eax, ecx
        ret             ; end of procedure
```

**Exercise**

**Write an assembly program to add two vectors `A` and `B` to obtain vector `C`. The program starts by getting the values of `A` and `B` from a user, then it prints out the vector `C`.**
*Solution*:
The general algorithm for this problem is as follows:

1. Read vector A
2. Read vector B
3. Computer C such that C[i] = A[i] + B[i]
4. Print vector C

Thus, we divide our problem into three sub-problems:

1. Reading a vector from a console
2. Adding two vectors
3. Printing out a vector to the console

Hence, the structure of our program is shown in Figure 10.

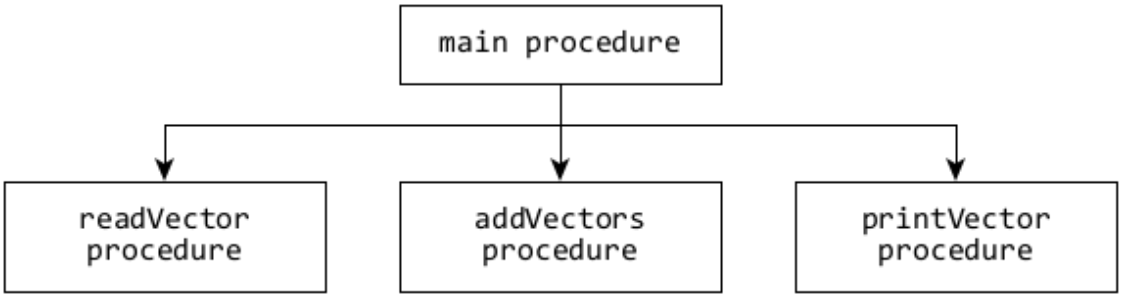



Figure 10

In this exercise, we are going to write a separate file for each procedure:

1. `readvector.asm`, which contains `readVector` procedure.
2. `addvectors.asm`, which contains addVectors procedure.
3. `printvector.asm`, which contains printVector procedure.
4. `main.asm`, which contains the main procedure.

```
;-----------------------------------------------
; File: readvector.asm
;-----------------------------------------------
            global  readVector
            extern  _scanf
            extern  _printf

            section .data
scan_fmt    db  "%d", 0

            section .code
;-----------------------------------------------
; Read a vector from a console user
;
; Receives:
;   EAX:    Input prompt
;   EDI:    A pointer to the vector
;   ECX:    Number of elements in the vector
;
; Returns:
;   A vector pointed by EDI register
;
; Requires: Nothing
;-----------------------------------------------
readVector:
            ; store EDI and ECX
            push    edi
            push    ecx

            ; print input prompt
            push    eax
            call    _printf
            pop     eax

            ; restore EDI and ECX
            pop     ecx
            pop     edi
top:
            push    ecx         ; store it again
            push    edi
            push    scan_fmt
            call    _scanf
```

```
;-----------------------------------------------
; File: addvectors.asm
;-----------------------------------------------
            global addVectors

            section .code

;-----------------------------------------------
; Add two vectors
;
; Receives:
;   EAX:    A pointer to the first vector
;   EBX:    A pointer to the second vector
;   EDI:    A pointer to the resulted vector
;   ECX:    Number of elements in the vector
;
; Returns:
;   The new vector pointed by EDI
;
; Requires: Nothing
;-----------------------------------------------
addVectors:
            ; save all data registers
            pushad
top:
            mov     edx, [eax]
            add     edx, [ebx]
            mov     [edi], edx

            add     eax, 4
            add     ebx, 4
            add     edi, 4
            loop    top

            ; restore all data registers
            popad

            ret
```

```
;-----------------------------------------------
; File: printvector.asm
;-----------------------------------------------
            global  printVector

            extern  _printf

            section .data
printf_fmt  db  "%d", 13, 10, 0

            section .code
;-----------------------------------------------
; Print out vector to the console screen
;
; Receives:
;   EAX:    Output prompt
;   EDI:    A pointer to the vector
;   ECX:    Number of elements in the vector
;
; Returns: Nothing
;
; Requires: Nothing
;-----------------------------------------------
printVector:
            ; store EDI and ECX
            push    edi
            push    ecx

            push    eax
            call    _printf
            pop     eax

            ; restore EDI and ECX
            pop     ecx
            pop     edi
top:
            push    ecx
            push    edi

            push    DWORD [edi]
            push    printf_fmt
            call    _printf
```

```
;-----------------------------------------------
; File: main.asm
;-----------------------------------------------
            global _main
            extern readVector
            extern printVector
            extern addVectors

            section .data
prompt1     db      "Enter vector A:", 13, 10, 0
prompt2     db      "Enter vector B:", 13, 10, 0
output_msg  db      "Vector C:", 13, 10, 0

VECSIZE     equ     5   ; number of elements in the vector

            section .bss
vecA        resd    VECSIZE
vecB        resd    VECSIZE
vecC        resd    VECSIZE

            section .code
;-----------------------------------------------
; The main procedure
;-----------------------------------------------
_main:
            mov     eax, prompt1
            mov     edi, vecA
            mov     ecx, VECSIZE
            call    readVector

            mov     eax, prompt2
            mov     edi, vecB
            mov     ecx, VECSIZE
            call    readVector

            mov     eax, vecA
            mov     ebx, vecB
            mov     edi, vecC
            mov     ecx, VECSIZE
            call    addVectors

            mov     eax, output_msg
```

### 3.5.1 How to compile and link the program

```
nasm -fwin32 main.asm -o main.obj
nasm -fwin32 readvector.asm -o readvector.obj
nasm -fwin32 addvectors.asm -o addvectors.obj
nasm -fwin32 printvector.asm -o printvector.obj
```

Then, you can link these object files along with C standard libraries:

```
gcc main.obj \
    readvector.obj \
    addvectors.obj \
    printvector.obj -o vector
```